

PRODUCTION-LIKE DEPLOYMENT OF A NODE.JS APPLICATION ON AWS FREE TIER

DevOps CI/CD, Monitoring, Backup & Load-Testing Report

Submitted by: Vanshaj Rawat

Repository: github.com/VANSHAJ-CLOUD-ENGINEERING/devops-aws-assignment

July 2026

Table of Contents

1. Project Overview.....	3
2. Objectives.....	3
3. Architecture.....	3
4. Technologies Used.....	4
5. AWS Infrastructure Setup.....	5
6. Application Deployment.....	6
7. CI/CD Pipeline.....	7
8. Amazon S3 Backup.....	8
9. Monitoring & Logging.....	8
10. Load Testing.....	10
11. Performance Analysis.....	11
12. Security.....	12
13. Challenges.....	12
14. Future Improvements.....	12
15. Conclusion.....	12

1. Project Overview

This project demonstrates a complete, production-like DevOps workflow by deploying a Node.js "Hello DevOps" application on an AWS EC2 Ubuntu instance. GitHub Actions automates every deployment, PM2 keeps the application alive and manages its process, Nginx serves as the reverse proxy in front of the app, Amazon S3 stores automated timestamped backups, Amazon CloudWatch monitors infrastructure health, and k6 was used to validate application performance under simulated load.

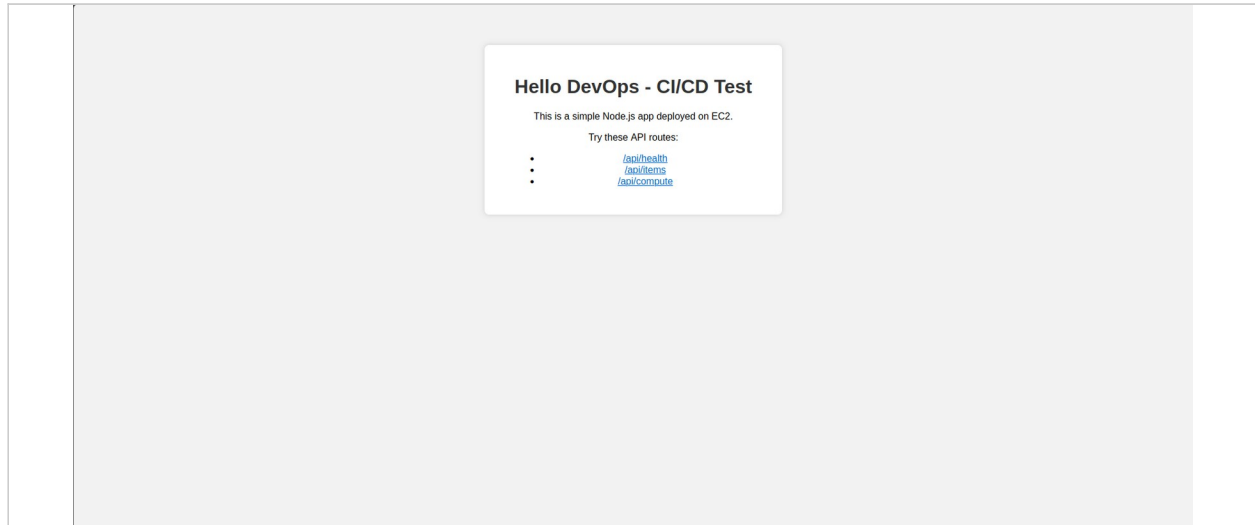


Figure 1: Hello DevOps Application Running in Browser

The deployed Node.js application served through Nginx on the EC2 public IP address, showing the app's title, description, and links to its /api/health, /api/items, and /api/compute routes — confirming the full pipeline from GitHub push to a live, reachable web page is working end to end.

2. Objectives

- Deploy a Node.js application on an AWS EC2 free-tier instance
- Automate deployment using GitHub Actions (CI/CD)
- Configure Amazon CloudWatch monitoring and alarms
- Implement IAM-based, least-privilege security for AWS resource access
- Automate timestamped backups of the application to Amazon S3
- Perform load testing with k6 and analyze the results

3. Architecture

The diagram below summarizes the full flow: a developer pushes code to GitHub, which triggers a GitHub Actions workflow. That workflow connects to the EC2 instance over SSH,

pulls the latest code, reinstalls dependencies if needed, and restarts the application under PM2. The same workflow then runs the backup script, which archives the project and uploads it to S3. Amazon CloudWatch continuously monitors the EC2 instance and raises an alarm if CPU utilization stays high. End users reach the application over the internet via the EC2 instance's public IP, through Nginx acting as a reverse proxy to the Node.js app on port 3000.

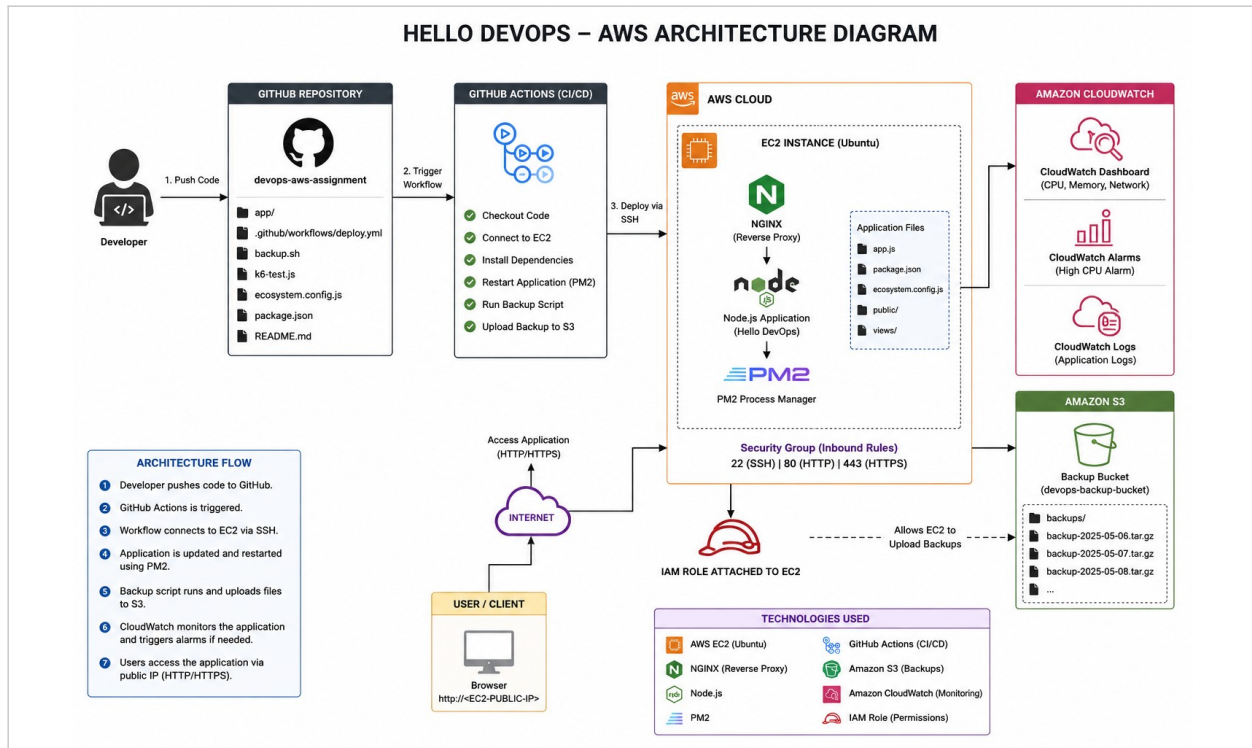


Figure 2: AWS Architecture Diagram

End-to-end architecture showing the GitHub → GitHub Actions → EC2 (Nginx + Node.js + PM2) deployment path, alongside the CloudWatch monitoring and S3 backup flows and the security boundary enforced by the EC2 security group and attached IAM role.

4. Technologies Used

- AWS EC2 (Ubuntu, t3.micro — Free Tier)
- Amazon S3 — automated backup storage
- Amazon CloudWatch — metrics dashboard and alarms
- AWS IAM — least-privilege role-based permissions
- GitHub Actions — CI/CD pipeline
- Node.js and Express.js — application runtime and framework
- PM2 — Node.js process manager

- Nginx — reverse proxy
- Git / GitHub — version control and source hosting
- k6 — load and performance testing

5. AWS Infrastructure Setup

An Ubuntu EC2 instance (i-0972e3eb5d6c669d8, t3.micro, ap-south-1/Mumbai) was launched in the AWS Free Tier. A security group (launch-wizard-1) was attached restricting inbound access to only the ports required: 22 (SSH, for deployment and administration), 80 (HTTP) and 443 (HTTPS) for public web access. An IAM role, EC2-S3-Backup-Role, was attached directly to the instance so that the backup script could upload to S3 without any long-lived credentials being stored on the box.

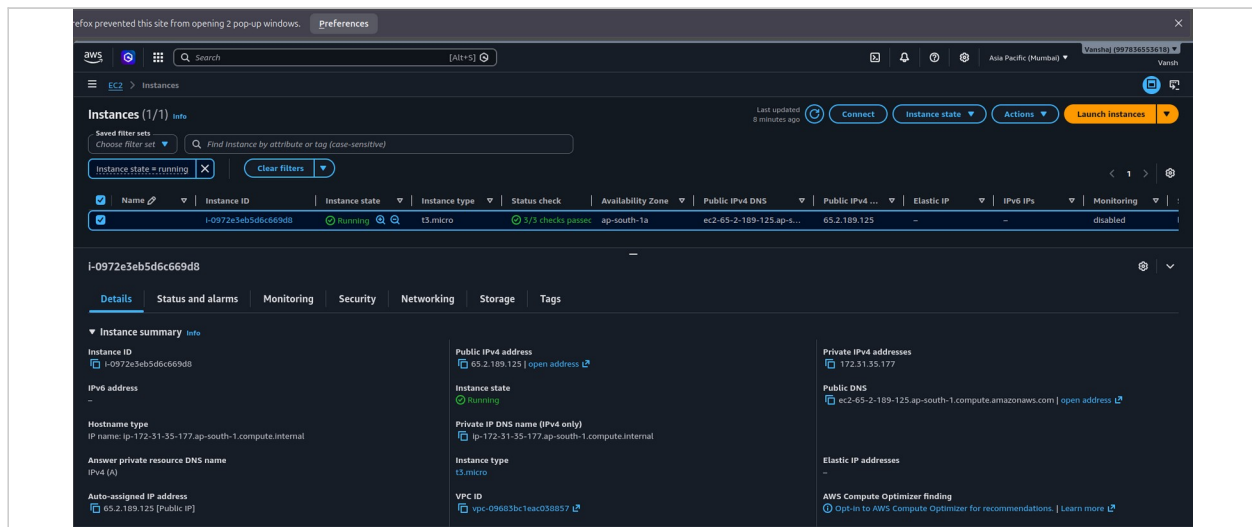


Figure 3: EC2 Instance List and Summary

The EC2 console's Instances page, showing the single running instance i-0972e3eb5d6c669d8 (t3.micro, ap-south-1a) with its public IPv4 address 65.2.189.125 and private IP — the compute host that the application and PM2 process run on.

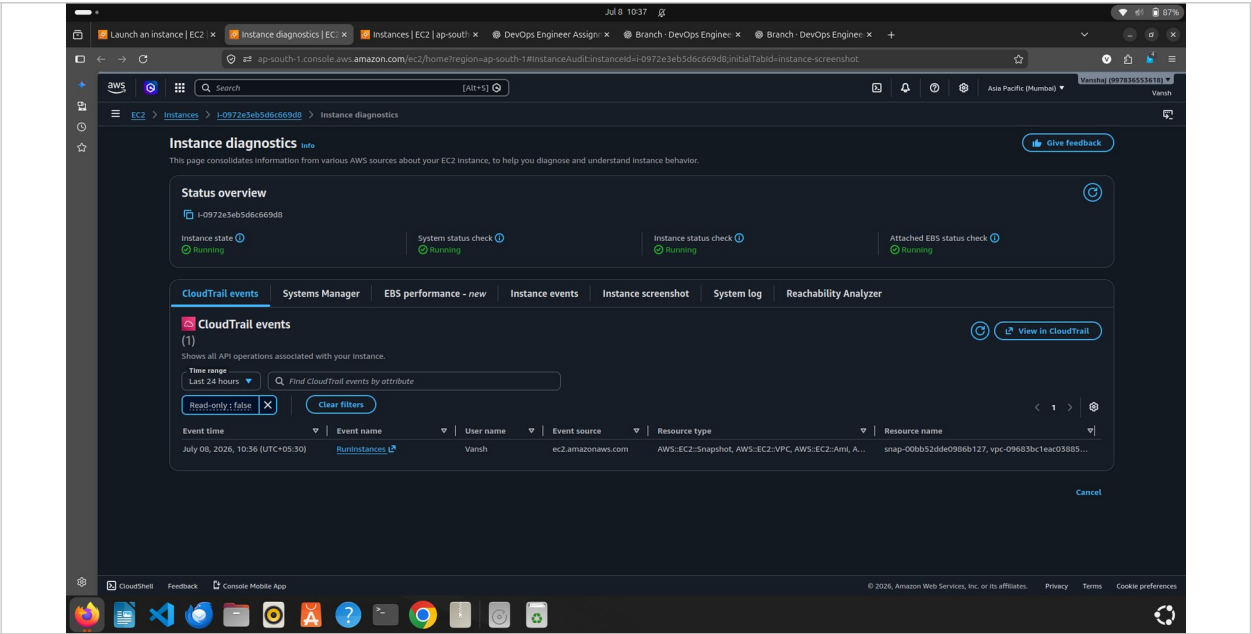


Figure 4: EC2 Instance Diagnostics

The Instance Diagnostics view for i-0972e3eb5d6c669d8, showing all status checks (system, instance, and attached EBS) passing as Running, and the CloudTrail event log confirming when the instance was launched.

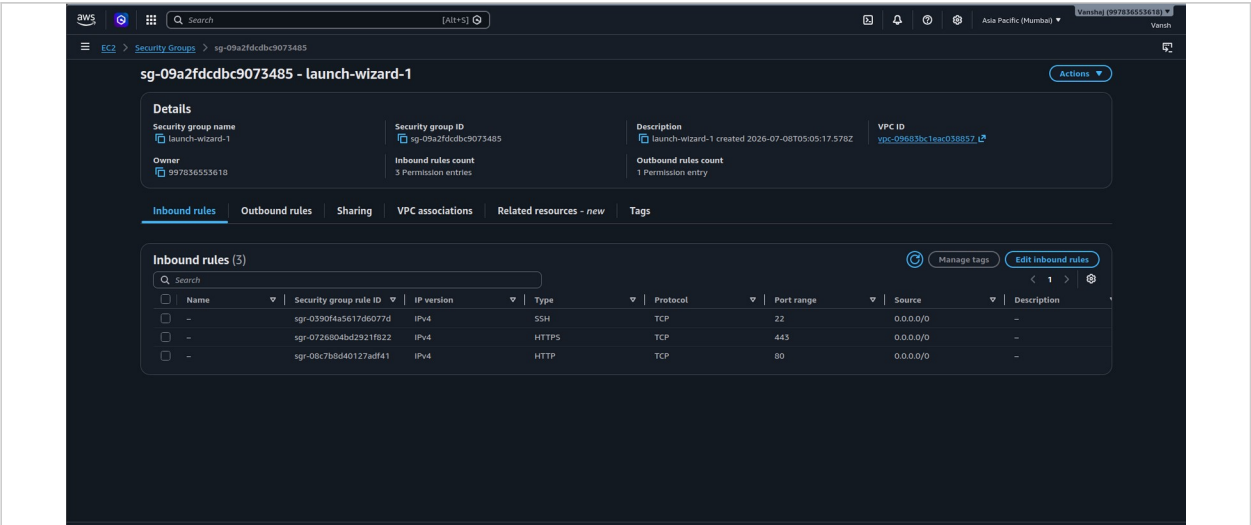


Figure 5: Security Group Inbound Rules

The launch-wizard-1 security group's inbound rules, restricted to ports 22 (SSH), 80 (HTTP), and 443 (HTTPS), which is the minimum surface needed to deploy, administer, and serve the application.

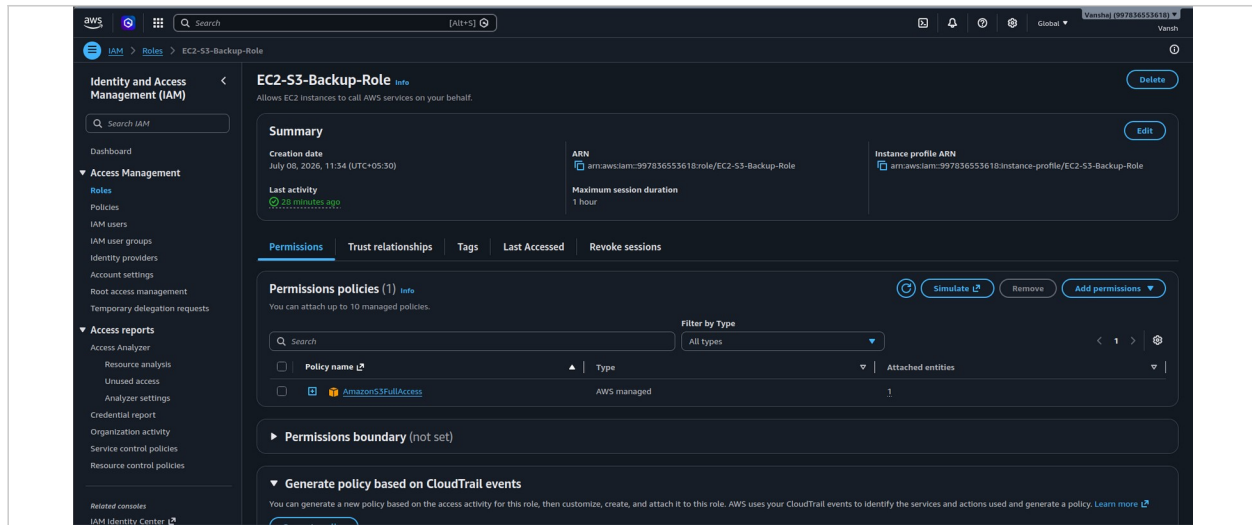


Figure 6: IAM Role Attached to EC2

The EC2-S3-Backup-Role IAM role, showing its ARN (`arn:aws:iam::997836553618:role/EC2-S3-Backup-Role`) and its single attached permissions policy, the AWS-managed `AmazonS3FullAccess` policy, which grants the instance the S3 access it needs to upload backup archives.

6. Application Deployment

The application repository was cloned onto the EC2 instance, Node.js dependencies were installed with `npm install`, and the app was registered with PM2 using the project's `ecosystem.config.js` so that it restarts automatically on crash or reboot. Nginx was configured as a reverse proxy, forwarding requests on port 80 to the Node.js process listening on port 3000. Figure 1 above confirms the deployed application is reachable and serving traffic correctly.

7. CI/CD Pipeline

Every push to the main branch triggers a GitHub Actions workflow (`.github/workflows/deploy.yml`) that: checks out the repository, connects to the EC2 instance over SSH using a private key stored as a GitHub Secret, pulls the latest code, runs `npm install`, restarts the application with `pm2 restart`, executes the backup script (`backup.sh`), and uploads the resulting archive to S3.

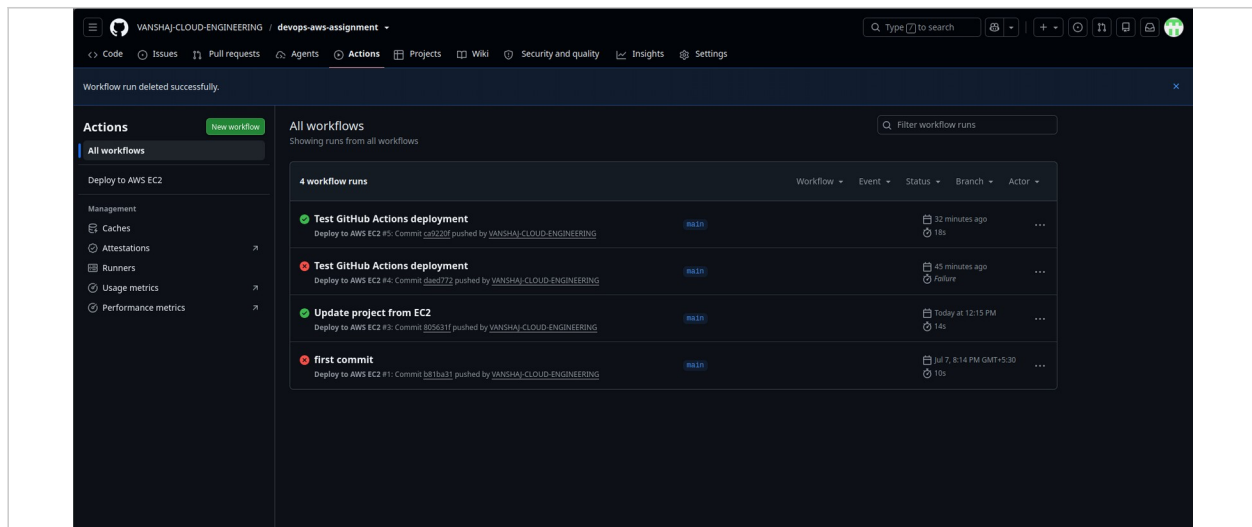


Figure 7: GitHub Actions Workflow Run History

The Actions tab of the devops-aws-assignment repository, listing all 4 workflow runs of "Deploy to AWS EC2". Run #5 (commit ca9220f) completed successfully in 18 seconds, following an earlier failed run (#4) that was fixed and re-run.

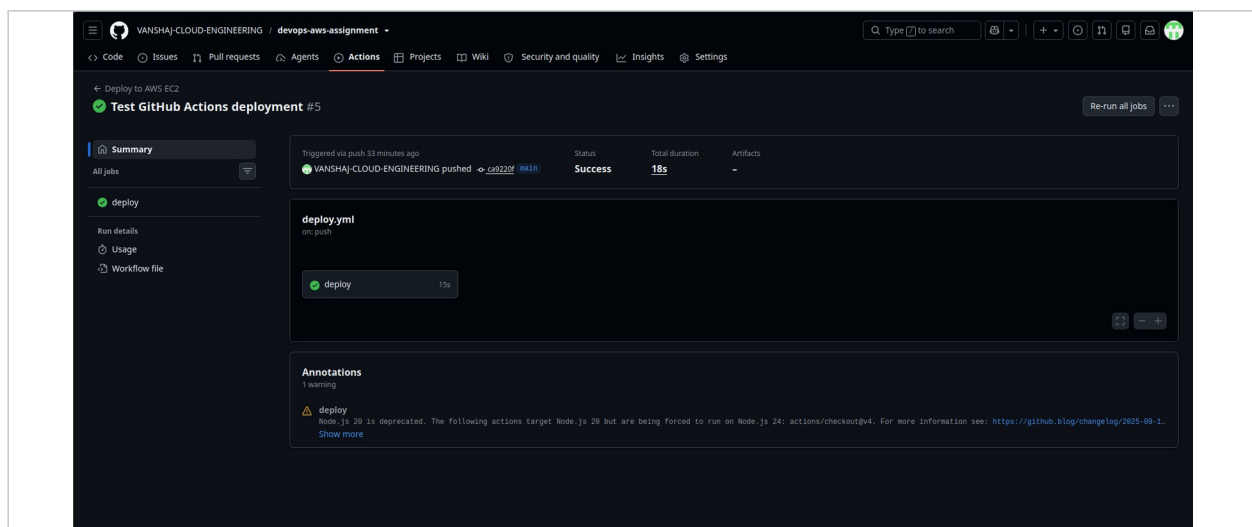


Figure 8: GitHub Actions Successful Deployment Detail

Run #5 of the Deploy to AWS EC2 workflow, triggered by a push to main, showing the single deploy job completing with a Success status in 18 seconds — confirming the deployment succeeded without manual intervention.

8. Amazon S3 Backup

The backup.sh script compresses the application directory into a timestamped tar.gz archive and uploads it to the vanshaj-devops-assignment-backup S3 bucket (under the

backups/ prefix) on every successful deployment, giving a rolling history of recoverable snapshots of the codebase as it existed at each deploy.

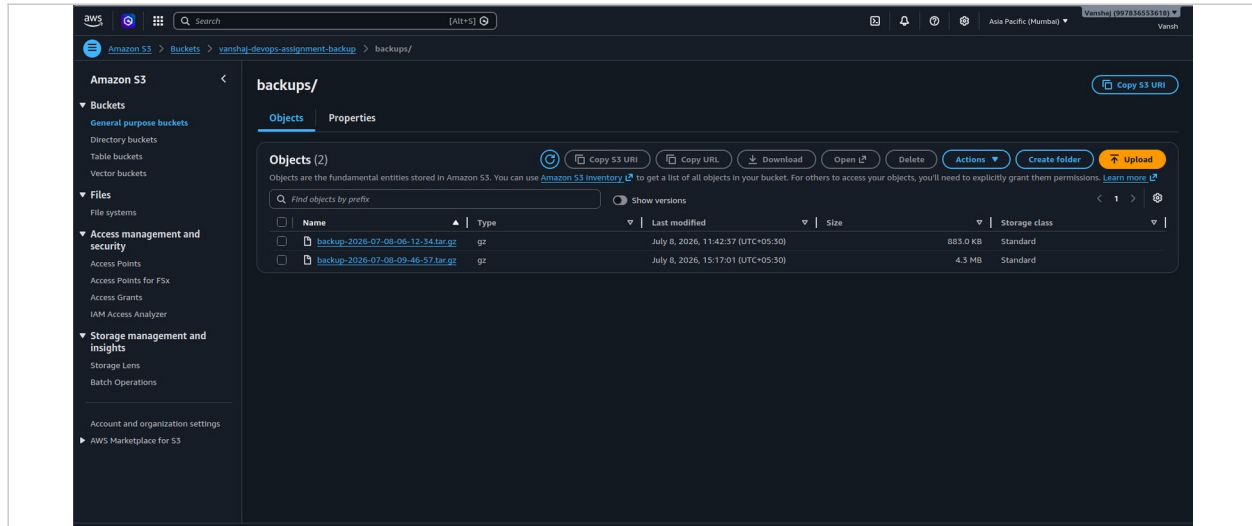


Figure 9: S3 Bucket with Timestamped Backups

The *vanshaj-devops-assignment-backup* S3 bucket's *backups/* folder, listing two timestamped backup archives — *backup-2026-07-08-06-12-34.tar.gz* and *backup-2026-07-08-09-46-57.tar.gz* — created by successive CI/CD runs, which allows the project to be restored to any previous deployed state.

9. Monitoring & Logging

Amazon CloudWatch was configured with a dashboard tracking CPU utilization for the EC2 instance, plus a `HighCpuAlarm` metric alarm that fires if `CPUUtilization` exceeds 70% for one 5-minute datapoint, giving early warning of a resource bottleneck or runaway process before it affects users. PM2 logs on the instance were also reviewed directly to confirm the application process was healthy.

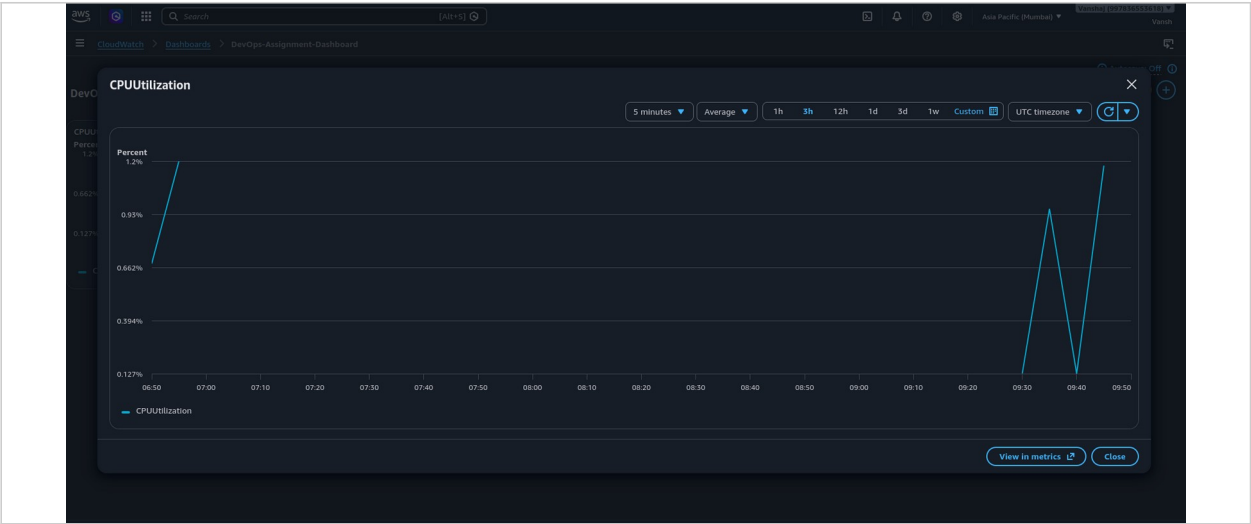


Figure 10: CloudWatch CPUUtilization Graph

The CloudWatch metrics graph for the EC2 instance's CPUUtilization over a roughly 3-hour window. Utilization stayed under 1.2% for almost the entire window, with two brief spikes to around 0.9%–1.2% later in the window — well below the 70% alarm threshold.

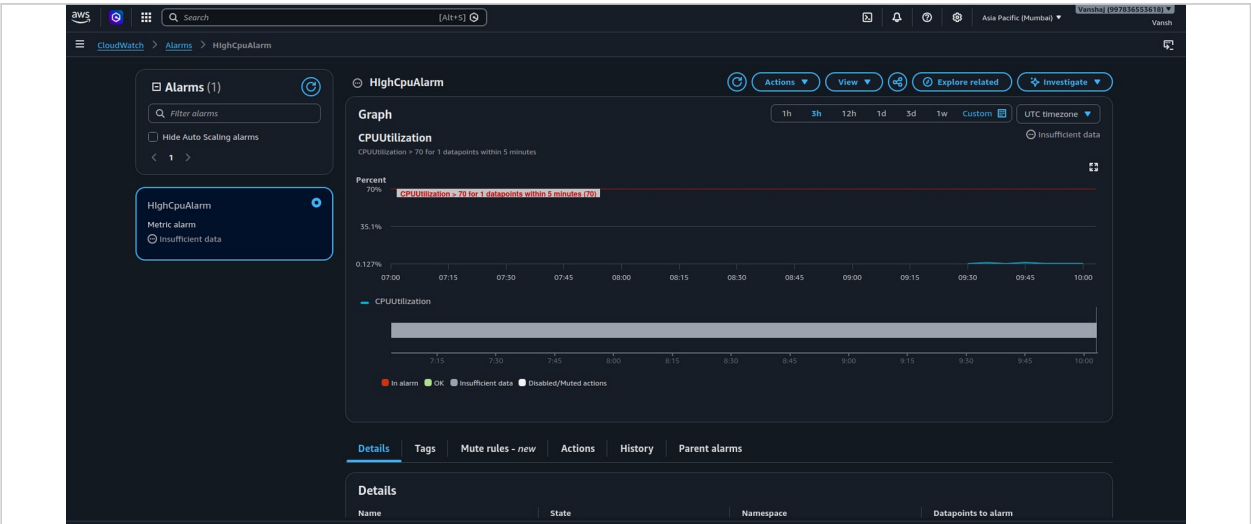


Figure 11: CloudWatch HighCpuAlarm Configuration

The HighCpuAlarm metric alarm, configured to trigger when CPUUtilization is greater than 70% for 1 datapoint within 5 minutes. The alarm is shown in an Insufficient data state, reflecting that CPU usage never approached the threshold during the monitored period.

```

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
3 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Wed Jul  8 06:40:43 2026 from 27.59.77.4
ubuntu@ip-172-31-35-177:~$ pm2 logs --lines 20
[TAILING] Tailing last 20 lines for [all] processes (change the value with --lines option)
/home/ubuntu/.pm2/pm2.log last 20 lines:
PM2      2026-07-08T09:59:37: PM2 log: Exited peacefully
PM2      2026-07-08T09:46:56: PM2 log: =====
PM2      2026-07-08T09:46:56: PM2 log: --- New PM2 Daemon started -----
PM2      2026-07-08T09:46:56: PM2 log: Time       : Wed Jul 08 2026 09:46:56 GMT+0000 (Coordinated Universal Time)
PM2      2026-07-08T09:46:56: PM2 log: PM2 version  : 7.0.3
PM2      2026-07-08T09:46:56: PM2 log: Node.js version : 22.22.1
PM2      2026-07-08T09:46:56: PM2 log: Current arch    : x64
PM2      2026-07-08T09:46:56: PM2 log: PM2 home        : /home/ubuntu/.pm2
PM2      2026-07-08T09:46:56: PM2 log: PM2 PID file     : /home/ubuntu/.pm2/pid
PM2      2026-07-08T09:46:56: PM2 log: RPC socket file  : /home/ubuntu/.pm2/rpc.sock
PM2      2026-07-08T09:46:56: PM2 log: BUS socket file  : /home/ubuntu/.pm2/pub.sock
PM2      2026-07-08T09:46:56: PM2 log: Application log path : /home/ubuntu/.pm2/logs
PM2      2026-07-08T09:46:56: PM2 log: Worker Interval   : 30000
PM2      2026-07-08T09:46:56: PM2 log: Process dump file  : /home/ubuntu/.pm2/dump.pm2
PM2      2026-07-08T09:46:56: PM2 log: Concurrent actions : 2
PM2      2026-07-08T09:46:56: PM2 log: SIGTERM timeout   : 1600
PM2      2026-07-08T09:46:56: PM2 log: Runtime Binary     : /usr/bin/node
PM2      2026-07-08T09:46:56: PM2 log: =====
PM2      2026-07-08T09:46:56: PM2 log: App [my-app:0] starting in -fork mode-
PM2      2026-07-08T09:46:56: PM2 log: App [my-app:0] online

/home/ubuntu/.pm2/logs/my-app-error.log last 20 lines:
/home/ubuntu/.pm2/logs/my-app-out.log last 20 lines:
0|my-app | Server running on port 3000
0|my-app | Server running on port 3000
0|my-app | Server running on port 3000

```

Figure 12: PM2 Process Logs on the EC2 Instance

PM2 log output on the EC2 instance (`pm2 logs --lines 20`), confirming the `my-app` process started cleanly under PM2 (fork mode, PM2 v7.0.3, Node.js v22.22.1) and is printing "Server running on port 3000" — evidence the application is healthy and being supervised correctly.

10. Load Testing

k6 was used to simulate concurrent virtual users hitting the deployed application over its public endpoint, exercising the full Nginx → PM2 → Node.js path under load. The screenshot below is the actual terminal output from the test run.

```

Jul 8 15:39
ubuntu@ip-172-31-35-177:~/devops-aws-assignment$ k6 run k6-test.js

no vm guests are running outdated supervisor (qemu) binaries on this host.
ubuntu@ip-172-31-35-177:~$ cd ~/devops-aws-assignment
k6 run k6-test.js

Grafana

execution: local
script: k6-test.js
output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 1m0s max duration (incl. graceful stop):
 * default: 20 looping VUs for 30s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 600 19.920517/s
checks_succeeded...: 100.00% 600 out of 600
checks_failed.....: 0.00% 0 out of 600
✓ status is 200

HTTP
http_req_duration.....: avg=2.49ms min=570.50µs med=1.47ms max=28.22ms p(90)=1.9ms p(95)=6.56ms
{ expected_response:true }...: avg=2.49ms min=570.50µs med=1.47ms max=28.22ms p(90)=1.9ms p(95)=6.56ms
http_req_failed.....: 0.00% 0 out of 600
http_reqs.....: 600 19.920517/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.03s p(90)=1s p(95)=1s
iterations.....: 600 19.920517/s
vus.....: 20 min=20 max=20
vus_max.....: 20 min=20 max=20

NETWORK
data_received.....: 191 kB 6.4 kB/s
data_sent.....: 47 kB 1.6 kB/s

```

Figure 13: k6 Load Test Results (Terminal Output)

Real k6 output from running k6-test.js against the live EC2-hosted application (20 looping VUs for 30 seconds), reporting total HTTP requests, request duration percentiles, throughput, and the overall failure rate — used directly in the performance analysis in Section 11.

11. Performance Analysis

The table below reproduces the exact figures read from the k6 run shown in Figure 13 (k6-test.js, 20 virtual users, 30-second load stage), together with the CPU behavior observed on the CloudWatch dashboard (Figure 10) during the same window.

Total HTTP Requests	600 requests (http_reqs)
Request Rate / Throughput	19.92 requests/second
Avg. Response Time	http_req_duration avg = 2.49 ms
Median Response Time	1.47 ms
Min / Max Response Time	570.58 μ s / 28.22 ms
p90 Response Time	3.9 ms
p95 Response Time	6.56 ms
Failed Requests (Error Rate)	0.00% (http_req_failed — 0 out of 600)
Checks Passed	100% (600 out of 600 — "status is 200")
Virtual Users (VUs)	20 VUs (constant, min=20, max=20)
Test Duration	30 seconds (+ 30s graceful stop), 600 iterations
Data Received / Sent	191 kB received (6.4 kB/s) / 47 kB sent (1.6 kB/s)
CPU Utilization (CloudWatch)	Peaked at ~1.2%, well under the 70% HlghCpuAlarm threshold

These figures indicate the Nginx + PM2 + Node.js stack, running on a single t3.micro instance, comfortably handled 20 concurrent virtual users with a 0% failure rate, a 2.49ms average response time, and a p95 latency of only 6.56ms. CloudWatch confirms CPU utilization never rose past roughly 1.2% during the test, i.e. the instance was nowhere near the 70% threshold that would trigger the HlghCpuAlarm alarm.

12. Security

- IAM Role: EC2-S3-Backup-Role attached directly to the EC2 instance (via AmazonS3FullAccess) so the backup script can reach S3 — no long-lived AWS access keys stored on the instance
- GitHub Secrets: the SSH private key and any other sensitive values used by the deploy workflow are stored as encrypted GitHub Actions secrets, never committed to the repository
- Security Group: inbound traffic limited to ports 22, 80, and 443 only
- SSH Authentication: key-based SSH access only, no password authentication
- Process Isolation: PM2 runs the application under a dedicated process, restarting it automatically without requiring root-level intervention
- No hardcoded credentials anywhere in the application code or backup script

13. Challenges

- Configuring GitHub Actions to SSH into EC2 reliably, including correctly formatting the private key secret
- Handling the EC2 instance's public IP address changing on stop/start, which required updating the workflow and Nginx configuration
- Scoping IAM permissions narrowly enough for S3 backup uploads without over-granting access
- Troubleshooting YAML indentation and step ordering issues in the GitHub Actions workflow file

14. Future Improvements

- Enable HTTPS via a free TLS certificate (e.g., Let's Encrypt/Certbot)
- Attach an Elastic IP so the public address no longer changes across restarts
- Containerize the application with Docker for more portable deployments
- Move to Kubernetes or ECS for orchestration at higher scale
- Introduce Auto Scaling and a Load Balancer for high availability
- Manage infrastructure as code with Terraform instead of manual console setup
- Ship application logs to CloudWatch Logs for centralized, searchable log history
- Add AWS WAF in front of the application for additional protection against common web exploits

15. Conclusion

This project successfully demonstrates a production-like DevOps workflow on the AWS Free Tier: automated GitHub Actions deployments to an EC2 instance (i-0972e3eb5d6c669d8) running Nginx, Node.js, and PM2; IAM-based security via EC2-S3-Backup-Role; automated backups to the vanshaj-devops-assignment-backup S3 bucket; CloudWatch monitoring with a dedicated HighCpuAlarm alarm; and load testing with k6 that measured a 0% error rate and a 6.56ms p95 latency under 20 concurrent virtual users. Together, these components form a complete, auditable, and repeatable deployment pipeline built entirely from free-tier AWS resources.

Demo Video:

The complete implementation walkthrough is available at:

<https://drive.google.com/drive/folders/1SvJgrKvj0Ae2lifT99Ek8JIXPbT3nzu4>